

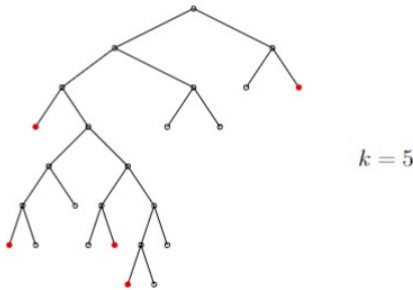
CS 473, Spring 2025

Homework 3 (due Feb 20 Thu 10am)

Instructions: As in previous homeworks.

Problem 3.1: (*Social distancing for koalas?*) We are given a binary tree T with n nodes, and a number $k \leq n$. (You may assume that every non-leaf node has exactly 2 children.) We want to pick a largest subset S of *leaves* such that every two different leaves in S have distance at least k . (The distance between two nodes u and v refers to the length of the (unique) path from u to v in T).

In the example below, a feasible subset S for $k = 5$ is shown in red, of size 5. (Turn the picture upside down if you are a koala :-)



Describe a dynamic programming algorithm to solve this problem. Include the following steps: (i) first define your subproblems precisely, (ii) then derive the recursive formula (including base cases) with brief justifications, (iii) specify the evaluation order, and (vi) analyze the running time and space as a function of n and k (think of k as smaller than n but k is not necessarily a constant). For this problem, you do not need to write pseudocode if your recursive formula and evaluation order are described clearly. And you do not need to write pseudocode to output an optimal solution; just the optimal size suffices.

[Hint: in defining subproblems, use an additional parameter $i \in \{0, \dots, k\}$ and add an extra constraint that all selected nodes must be of distance at least i from the root...]

Solution:

(i) Subproblems:

Define $C(u, i)$ as the maximum number of leaves that can be selected in the subtree rooted at u , such that selected leaves are at least distance k apart, and every selected leaf is at least distance i from u . Want $C(\text{root}, 0)$.

(ii) Recursive formula:

Let L and R be the left and right children of u . For leaf nodes (base case):

$$C(u, i) = \begin{cases} 1, & \text{if } i = 0 \\ 0, & \text{if } i > 0 \end{cases}$$

For internal nodes:

$$C(u, i) = \max_{\substack{0 \leq l, r \leq k+1-i \\ 2i+l+r \geq k}} \{C(L, i-1+l) + C(R, i-1+r)\}$$

Since $C(u, i + \Delta) \leq C(u, i)$ for $\Delta > 0$, $C(u, i)$ only takes $O(k)$ time to compute.

(iii) Evaluation order:

Use a bottom-up approach via postorder traversal. Compute C values for child nodes before their parent.

(iv) Time and space complexity:

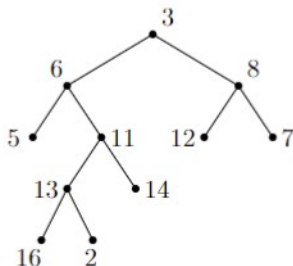
Each node has $O(k)$ states, and each state is computed in $O(k)$ time.

Total time complexity: $O(nk^2)$

Total space complexity: $O(nk)$

Problem 3.2: (*Turning a sequence into a tree*) We are given a sequence of n numbers $\langle a_1, \dots, a_n \rangle$. We want to compute a proper binary tree T with nodes $a_1, \dots, a_n$ such that the *inorder traversal* is precisely $\langle a_1, \dots, a_n \rangle$, while minimizing the cost function $c(T) = \sum_{(a_i, a_j) \text{ is an edge of } T} |a_i - a_j|$. (A proper binary tree refers to a rooted tree where each internal node of T has exactly 2 children.)

For example, for the input sequence $\langle 5, 6, 16, 13, 2, 11, 14, 3, 12, 8, 7 \rangle$, one feasible solution is the following tree, with cost $|3 - 6| + |3 - 8| + |6 - 5| + |6 - 11| + |8 - 12| + |8 - 7| + |11 - 13| + |11 - 14| + |13 - 16| + |13 - 2| = 38$ (we do not claim that this is optimal).



Describe a dynamic programming algorithm to solve this problem. For full credit, the running time should be at most $O(n^4)$. Include the following steps: (i) first define your subproblems precisely, (ii) then derive the recursive formula (including base cases) with brief justifications, (iii) specify the evaluation order, and (vi) analyze the running time and space. For this problem, you do not need to write pseudocode if your recursive formula and evaluation order are described clearly. And you do not need to write pseudocode to output an optimal solution; just the optimal cost suffices.

[Hint: define $C(i, j, m)$ to be the minimum cost over all binary trees T for the subsequence $\langle a_i, a_{i+1}, \dots, a_j \rangle$ with the extra constraint that the root is a_m .]

Solution:

(i) Subproblems:

Define $C(i, j, m)$ to be the minimum cost over all binary trees T for the subsequence $\langle a_i, a_{i+1}, \dots, a_j \rangle$ with root a_m . Want $\min_{1 \leq m \leq n} C(1, n, m)$.

(ii) Recursive formula:

$$C(i, j, m) = \min_{i \leq l < m < r \leq j} \{C(i, m-1, l) + C(m+1, j, r) + |a_l - a_m| + |a_r - a_m|\}$$

Base case: $C(i, i+1, m) = |a_i - a_{i+1}|$ for $1 \leq i < n$.

(iii) Evaluation order:

Increase j from 1 to n , for each j , decrease i from j to 1, for each i, j , evaluate m from i to j .

(iv) Time and space complexity:

Iteration takes $O(n^3)$ steps, each step takes $O(n)$ time to evaluate.

Total time complexity: $O(n^4)$.

Total space complexity: $O(n^3)$.

Problem 3.3: We are given a directed graph $G = (V, E)$ with n vertices and m edges (with $m \geq n$), where each edge e has a *weight* $w(e)$ and each vertex v has a *penalty* $p(v)$.

In addition, we are given a *source* vertex $s \in V$, and a number $K \leq n$.

We want to find a path π from s to some vertex v using at most K edges, minimizing $\text{cost}(\pi) := (\sum_{e \in \pi} w(e)) + p(v)$ (i.e., the weight of the path π plus the penalty at v). Note that in this problem, we want to output not just the optimal cost but also an optimal path.

- (a) (50 pts) Describe a dynamic programming algorithm to solve this problem in $O(mK)$ time and $O(nK)$ space.

Include *all* of the following steps: (i) first define your subproblems precisely, (ii) then derive the recursive formula (including base cases) with brief justifications, (iii) specify the evaluation order, (iv) write pseudocode to output the optimal cost, (v) write pseudocode to output an optimal path, and (vi) analyze the running time and space.

[Hint: define $C(u, k)$ to be the minimum cost of a path from u using at most k edges. . .]

- (b) (50 pts) Design and analyze a slightly slower but more space-efficient algorithm to solve this problem in $O(mK \log K)$ time and $O(n \log K)$ space.

[Hint: use divide-and-conquer in combination with your algorithm from (a) (somewhat similar to the space-saving trick from class but with a different recurrence).]

Solution:

(a)

- (i) Subproblems:

Define $C(i, k)$ to be the minimum cost of a path to u using at most k edges.

Want $\min_{v \in V} \{C(v, K) + p(v)\}$.

- (ii) Recursive formula:

$$C(i, k) = \min_{e \in E} \{C(i, k), C(j, k-1) + w(e)\}$$

where e is an edge from j to i .

Base case:

$$C(i, 0) = \begin{cases} 0, & \text{if } i = s, \\ +\infty, & \text{otherwise.} \end{cases}$$

- (iii) Evaluation order:

Increase k from 1 to K , for each k , iterate through all $e \in E$ to evaluate C .

- (iv) Optimal cost:

```
C = array of size (n, 2) initialized to inf
C[s][0] = 0
P = array of size (n, K) recording previous step
for k = 1 to K:
    for e in E:
```

```

    # suppose e is an edge from j to i
    evaluate C[i][1] according to (ii)
    update P if there is a better path to i
  roll C
for v in V:
  for k = 1 to K:
    find min cost according to (i)

```

(v) Optimal path:

```

v = vertex with minimum cost from (iv)
path = array storing the optimal path
for k = K to 1:
  add P[v][k] to the beginning of path
  v = P[v][k]
add s to the beginning of path
return path

```

(vi) Time and space complexity:

Iteration takes $O(mK)$ steps, each step takes $O(1)$ time to evaluate. Finding minimum cost takes extra $O(nK)$ time, which is smaller than $O(mK)$.

Total time complexity is (mK) .

Total space complexity is (nK) .

(b) Define $D(L, R, C_L) = C_R$ to be the recursive routine with state C_L from step L to R where $1 \leq L < R \leq K$, $P_k(i)$ to be the parent pointer at step k .

One observation is we do not need $P_{i+\Delta}$ or $C_{i+\Delta}$ ($\Delta > 0$) while reconstructing step i .

Suppose $mid = \frac{L+R}{2}$, recursively compute $D(L, mid, C_L)$ then $D(mid, R, C_{mid})$, record parent pointer $P_{mid}(i)$ and C_{mid} on the right side only (e.g. $C_8, C_{12}, C_{14}, C_{15}$ if $n = 16$), taking $O(n \log K)$ space in total.

After computing the optimal cost, reconstruct path according to P and C recorded at each recursive depth, recompute left sides recursively to replace P and C on the right side (e.g. replace C_{15} with C_{13} at depth 4), in order to reconstruct full path on both sides. Time complexity is $O(nK)$ at each depth. Total time complexity is $O(nK \log K)$.